@k4coding

TWO APPLIES. ONE BROKEN STATE.

Terraform state locking, explained in 9 slides.

```
2x terraform apply = lost changes
```

Swipe >

THE SETUP

Two engineers, or two CI jobs, run terraform apply on the same state file at the same time.

Swipe >

WHAT BREAKS

Without a lock, both read the same state.

Both write it back.

The last write wins, and changes from the other run disappear from state.

THE SYMPTOMS

"Already exists" errors on the next apply.

Resources that exist in the cloud but not in state.

Plans that no longer match reality.

FIX 1: REMOTE STATE

Move state to a remote backend that supports locking: S3, GCS, azurerm or HCP Terraform.

Never keep shared state on a laptop.

FIX 2: LOCK IT

On S3, Terraform 1.10+ can lock with a lock file. No DynamoDB table needed.

```
backend "s3" {
  bucket       = "tf-state"
  key          = "prod/state"
  region       = "ap-south-1"
  use_lockfile = true
}
```

FIX 3: ONE APPLY

In CI, allow only one apply per environment at a time.

In GitHub Actions, a concurrency group does this.

```
concurrency:
  group: terraform-prod
  cancel-in-progress: false
```

FORCE-UNLOCK RULE

A stuck lock? Confirm no apply is still running first.

Only then run terraform force-unlock with the lock ID.

SUMMARY

Remote backend with locking.

use_lockfile on S3 (Terraform 1.10+).

One apply per environment in CI.

Check before force-unlock.

Follow for one DevOps fix every day.